

ML Interview Preparation — Complete Checklist, Scenario Notes & Case Studies

Expanded study sheet covering the ML concepts already studied, the pollution/AQI scenario, and additional scenario-based case studies likely to test structured ML reasoning in a Data Science / Analytics / AI Engineer interview.

1. ML Fundamentals Checklist

Overfitting, Underfitting & Bias–Variance

- Overfitting: model learns training data/noise too closely; training performance is strong but unseen-data performance is poor.
- Underfitting: model is too simple and performs poorly even on training data.
- High bias $\rightarrow$ underfitting. High variance $\rightarrow$ overfitting.
- High variance means the learned model can change substantially when the training data changes slightly.
- Reduce variance with simpler models, regularization, more data, feature selection, or tree pruning.
- Reduce bias with more model capacity, useful features, less regularization, or better optimization when optimization is the issue.

Regularization

- L2: $J(w) = \text{Loss}(w) + \lambda w^2/2$. Its gradient adds λw , shrinking large weights more strongly.
- L1: $J(w) = \text{Loss}(w) + \lambda |w|$. It can drive some weights exactly to zero, giving feature-selection behavior.

Regression vs Classification

- Regression predicts a continuous numeric target. Classification predicts one or more class labels.
- Both are supervised learning because training data contains target values/labels.

Regression Metrics

- MAE = $(1/n) \sum |y_i - \hat{y}_i|$. Average absolute prediction error; highly interpretable.
- MSE = $(1/n) \sum (y_i - \hat{y}_i)^2$. Squaring makes large errors matter more.
- RMSE = $\sqrt{(1/n) \sum (y_i - \hat{y}_i)^2}$. Same target units and gives larger errors more weight.
- $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$. Compares the model with mean prediction.
- $R^2 = 1$ perfect, 0 mean-baseline performance, and can be negative when worse than the mean baseline.
- MAE and R^2 answer different questions: MAE gives error in target units; R^2 describes improvement relative to mean prediction.

Logistic Regression

- Compute $z = w^T x + b$, pass it through sigmoid $p = 1/(1+e^{-z})$, then optimize binary cross-entropy.
- BCE = $-[y \log(p) + (1-y) \log(1-p)]$.
- Sigmoid gives a value between 0 and 1 that can be interpreted as a positive-class probability under the model.

Confusion Matrix & Classification Metrics

- TP: predicted positive and actually positive. FP: predicted positive but actually negative.
- FN: predicted negative but actually positive. TN: predicted negative and actually negative.
- Precision = $TP/(TP+FP)$. Recall = $TP/(TP+FN)$. F1 is the harmonic mean.

- High recall means fewer false negatives. High precision means fewer false positives.
- $TPR = \text{Recall}$. $FPR = FP/(FP+TN)$.

ROC-AUC & PR-AUC

- ROC evaluates TPR versus FPR across classification thresholds.
- AUC summarizes ranking/separation ability: 1 is perfect, about 0.5 is random ranking.
- PR-AUC is often more informative for severe class imbalance because it focuses on precision and recall.

Decision Trees

- Trees learn splits on features to reduce impurity/error. They can handle classification and regression.
- $Gini = 1 - \sum p_i^2$. $Entropy = -\sum p_i \log_2(p_i)$. Information gain = parent impurity – weighted child impurity.
- Common controls: max_depth, min_samples_split, min_samples_leaf, criterion, max_leaf_nodes, max_features, ccp_alpha.
- Pruning can reduce overfitting.

Random Forest / Bagging / Boosting

- Random Forest builds many diverse trees using bootstrap samples and random feature subsets; classification uses voting and regression uses averaging.
- Bagging trains learners independently and combines them.
- Boosting trains sequentially so later learners focus on previous errors.
- AdaBoost reweights difficult examples. Gradient Boosting fits corrections/residuals. XGBoost adds optimized training and regularization. LightGBM emphasizes efficient large-scale training.

Other Core Models

- KNN: distance-based lazy learner; scale features. Small K → higher variance; large K → higher bias.
- Naive Bayes: $P(C|X) \propto P(X|C)P(C)$, with naive conditional independence across features.
- SVM: maximum-margin classifier; support vectors are closest points. Kernels enable nonlinear boundaries. SVR handles regression.
- K-means: assign points to nearest centroid, recompute means, repeat. Points do not move; centroids do.
- PCA: unsupervised dimensionality reduction; PC1 is a direction/weighted combination maximizing projected variance.

Scaling / Selection / Leakage

- Standardization: $x' = (x - \mu) / \sigma$. Min-max: $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$. Fit preprocessing on training data only.
- Scaling is especially important for KNN, K-means, SVM, PCA, and many neural-network workflows.
- Feature selection chooses existing features; feature engineering creates new features.
- Leakage occurs when information unavailable at prediction time influences training/model selection.

Train / Validation / Test & CV

- Train learns model parameters. Validation supports model selection/tuning. Test is the final untouched evaluation.
- K-fold CV is generally performed within the training portion during model selection.
- Typical workflow: split → CV/tuning → select model → retrain selected model on all training data → evaluate once on untouched test.

Imbalance / Missing Values / Outliers

- Accuracy can be misleading when one class dominates; e.g. 95% majority class can give 95% accuracy with a useless majority-only classifier.

- Possible imbalance approaches: class weights, over/undersampling, SMOTE, threshold tuning.
- Choose metrics by business cost: recall when missing positives is costly, precision when false alarms are costly, F1 when both matter, PR-AUC for severe imbalance.
- Missing values: drop when appropriate or impute with mean/median/mode or advanced methods. Fit imputation on training data only.
- Investigate outliers with IQR or z-score. Do not automatically delete them; determine whether they are errors or genuine observations.

2. Universal Framework for Scenario Questions

- 1. Define the business problem.** What exactly are we predicting or optimizing?
- 2. Define X and y.** Identify features and target. Decide regression, classification, clustering, forecasting, or anomaly detection.
- 3. Understand the data.** Data size, feature types, missing values, outliers, class balance, duplicates, and time dependence.
- 4. Split correctly.** Random split for i.i.d. data; time-based split for forecasting/time-dependent data. Avoid leakage.
- 5. Establish a baseline.** Mean/recent-value baseline for regression or majority/rule-based baseline for classification.
- 6. Engineer useful features.** Lags, rolling statistics, ratios, interactions, temporal features, and domain features—only from information available at prediction time.
- 7. Start simple, then increase complexity.** Linear/logistic regression or a simple tree baseline, then Random Forest/Gradient Boosting when nonlinearities/interactions matter.
- 8. Pick metrics from business cost.** MAE/RMSE/R² for regression; precision/recall/F1/ROC-AUC/PR-AUC for classification.
- 9. Validate and compare.** Use validation or CV on training data, then one final evaluation on untouched test data.
- 10. Explain and monitor.** Feature importance is predictive, not causal. In production, monitor performance, drift, missingness, latency, and data quality.

3. AQI / Pollution Case Study — Complete Interview Reasoning

Scenario: A city government wants to predict the next day's AQI using historical AQI/pollution information and environmental or activity-related features.

Problem formulation

X can contain historical AQI, temperature, humidity, wind speed, traffic volume, industrial activity, rainfall, day of week, and past pollutant measurements. y is next-day AQI, so this is a regression problem.

Because this is forecasting, use a time-based train/validation/test strategy. A random split can leak future patterns into training. Keep a simple recent-value or historical-mean baseline.

Feature interpretation

Historical AQI and past pollutant measurements may overlap, but they are not necessarily identical. AQI can represent the overall index, while pollutant measurements can represent PM2.5, PM10, NO₂, SO₂, CO, etc.

Temperature and humidity are predictors rather than AQI measurements. Weather can affect dispersion and accumulation. Day of week can capture weekly traffic/industrial patterns. Traffic volume means road-traffic volume in this scenario.

Do not claim that a feature causes a percentage of AQI from feature importance. Predictive importance is not causality.

Feature engineering

Use leakage-safe lag features such as AQI 1/2/7 days ago, rolling 3/7-day means, temporal features, and justified interactions such as traffic × wind or industrial activity × wind.

Model choice

Start with a baseline and linear regression. Then compare tree ensembles such as Random Forest or Gradient Boosting because AQI relationships can be nonlinear and involve interactions.

Evaluation

MAE is a strong primary metric when average absolute error is easy to interpret. RMSE is useful when large errors deserve extra penalty. R^2 gives performance relative to a mean-prediction baseline.

Feature importance

Train the normal Random Forest first. Its trees make error/impurity-reducing splits; features contributing to useful splits can receive higher built-in importance. Permutation importance instead measures how much performance worsens when one feature is shuffled. Neither proves causality.

Q: Why not simply drop temperature because it is not AQI?

A: Because a predictor does not need to be the target itself. Temperature can affect pollution behavior and may improve prediction. Validate it rather than dropping it from intuition alone.

Q: Why can day of week matter?

A: Traffic and industrial activity can have weekly patterns, so day of week can carry predictive information.

Q: What if traffic seems less important than expected?

A: That is fine. Domain intuition gives a hypothesis; the data and validation should decide predictive usefulness.

Q: What if MAE is 12—is that good?

A: Not by itself. Compare it with a baseline, target variability, operational tolerance, and the cost of prediction errors.

Q: What if the interviewer asks for the complete answer?

A: Say: 'I would formulate it as a next-day AQI regression problem, use historical pollution and weather/activity features, create leakage-safe lag and rolling features, use time-based validation, start with a simple baseline, then compare linear and tree-based models. I would use MAE primarily, RMSE if large errors matter, and R^2 as a relative-to-baseline measure.'

4. Case Study — Customer Churn Prediction

Scenario: A telecom company wants to predict which customers will churn in the next 30 days.

Q: What is X and y?

A: X contains customer usage, tenure, plan, payment behavior, support interactions, recent complaints, etc. y is churn: yes/no. This is binary classification.

Q: Which model would you start with?

A: Start with logistic regression as an interpretable baseline, then compare Random Forest or Gradient Boosting for nonlinear patterns.

Q: What metric?

A: If losing a customer is costly, recall may matter because missing a true churner is costly. If retention offers are expensive, precision also matters. F1 can balance both.

Q: What is a major leakage risk?

A: Using information created after the prediction cutoff, such as a cancellation request already submitted or a future invoice/payment outcome.

Q: How would you handle imbalance?

A: Use class weights, appropriate resampling such as SMOTE when suitable, and evaluate precision/recall/F1 or PR-AUC rather than accuracy alone.

Q: How would you turn predictions into action?

A: Rank customers by churn probability and target retention actions within budget. Then measure both model performance and business lift/cost.

5. Case Study — Fraud Detection

Scenario: A bank wants to detect fraudulent transactions in real time.

Q: Regression or classification?

A: Binary classification: fraud vs legitimate.

Q: What makes this case tricky?

A: Severe class imbalance, changing fraud patterns, real-time latency, and asymmetric costs of false positives vs false negatives.

Q: Which metric?

A: Recall is important for catching fraud, but precision matters because false alarms can block legitimate customers. PR-AUC and threshold-specific precision/recall are useful.

Q: Random split or time split?

A: Prefer time-aware validation because fraud patterns and attacker behavior change over time and future information must not leak backward.

Q: What features could matter?

A: Transaction amount, merchant/category, time, location consistency, device history, transaction frequency, velocity features, and deviations from the customer's normal behavior.

Q: What is a useful feature engineering example?

A: Number/value of transactions in the previous hour/day, but calculate it only from information available before the current transaction.

Q: Why not optimize only accuracy?

A: A model can predict every transaction as legitimate and still have very high accuracy while detecting almost no fraud.

6. Case Study — Loan Default / Credit Risk

Scenario: A lender wants to predict whether an applicant will default.

Q: What is the target?

A: Default yes/no, so binary classification.

Q: What features?

A: Income, debt-to-income ratio, credit history, repayment history, loan amount, tenure, employment information, and other legally/operationally appropriate variables.

Q: Which metric?

A: There is no universal answer. Recall may matter if missing risky borrowers is costly; precision may matter if unnecessary rejection is costly. Discuss the business cost.

Q: What leakage could happen?

A: Using post-loan information such as missed payments that occurred after approval to predict whether the applicant would default at approval time.

Q: What extra concern should you mention?

A: Fairness, explainability, regulatory requirements, and monitoring are important in high-impact lending decisions.

7. Case Study — Sales / Demand Forecasting

Scenario: A retailer wants to predict product demand for the next week.

Q: Regression or classification?

A: Demand is numeric, so regression/forecasting.

Q: Important features?

A: Past sales, promotions, price, holidays, seasonality, day of week, inventory availability, weather where relevant, and product/store information.

Q: Why is inventory important to interpret carefully?

A: Observed sales can be lower than true demand when the product was out of stock. The model should distinguish demand from constrained sales where possible.

Q: How do you validate?

A: Use chronological train/validation/test splits and compare against a simple recent-value or seasonal baseline.

Q: What feature engineering?

A: Lagged sales, rolling means, seasonal indicators, promotion flags, and other leakage-safe historical summaries.

Q: Which metric?

A: MAE is interpretable; RMSE penalizes large misses; choose based on inventory/business costs.

8. Case Study — Medical Disease Prediction

Scenario: A hospital wants to predict whether a patient has a disease from test results and clinical features.

Q: Classification or regression?

A: Usually binary classification if the output is disease/no disease.

Q: Precision vs recall?

A: If missing a disease is very costly, prioritize recall. If false alarms lead to expensive/invasive follow-up, precision becomes more important.

Q: Would accuracy be enough?

A: No. Inspect the confusion matrix and precision/recall/F1; ROC-AUC or PR-AUC can evaluate ranking across thresholds.

Q: Why might threshold 0.5 be wrong?

A: The best threshold depends on the cost of false negatives vs false positives and the operating context.

Q: What should you mention beyond the model?

A: Calibration, clinical validation, data quality, fairness, explainability, privacy, and careful deployment/monitoring.

9. Case Study — Recommendation System

Scenario: An e-commerce platform wants to recommend products to users.

Q: What could the target be?

A: Depending on the design: click, purchase, rating, watch time, or another engagement signal.

Q: What data?

A: User-item interactions, item metadata, recent behavior, popularity, and contextual information.

Q: What is the cold-start problem?

A: New users or new items have little or no interaction history, so collaborative signals are weak.

Q: How could you approach it?

A: Start with popularity/rule-based recommendations, then use content-based, collaborative, or hybrid approaches as data supports them.

Q: What should you evaluate?

A: Offline ranking metrics such as precision@K/recall@K can be used, but online A/B testing is important because offline gains may not translate to business outcomes.

10. Case Study — Anomaly Detection

Scenario: A manufacturing company wants to detect abnormal machine behavior before failures.

Q: Supervised or unsupervised?

A: If labeled failures are available, supervised classification can work. Without reliable labels, anomaly detection methods can be appropriate.

Q: What features?

A: Sensor readings, rates of change, rolling statistics, temperature/pressure/current, operating mode, and historical machine behavior.

Q: What challenge?

A: Normal behavior can vary by machine and operating mode, so context and baselines matter.

Q: How would you evaluate?

A: If labels exist, use precision/recall and business costs. If labels do not exist, evaluate alerts through expert review and eventual failure outcomes.

11. Case Study — Sentiment / Text Classification

Scenario: A company wants to classify customer reviews as positive, neutral, or negative.

Q: What is the task?

A: Multiclass classification.

Q: Basic approach?

A: Tokenize text, create a representation such as TF-IDF or embeddings, then train a classifier. A modern transformer can be used when justified.

Q: What metric?

A: Accuracy can work with balanced classes; otherwise inspect per-class precision/recall/F1 and macro-F1.

Q: What common issue?

A: Class imbalance, ambiguous language, sarcasm, domain-specific vocabulary, and data leakage from duplicate or near-duplicate reviews.

12. Rapid-Fire Scenario Questions You Should Be Ready For

Q: Your model has 98% accuracy but 2% positive cases. Is that good?

A: Not necessarily. Compare against the majority baseline and inspect precision, recall, F1, PR-AUC, and the confusion matrix.

Q: Train score is 99%, test score is 70%. What does it suggest?

A: Likely overfitting/high variance. Consider simpler models, regularization, more data, feature selection, or pruning.

Q: Both train and test scores are poor. What next?

A: Suspect underfitting/high bias, weak features, excessive regularization, or an optimization/data-quality issue.

Q: A feature is highly important. Does it cause the target?

A: No. Feature importance is predictive association, not causal proof.

Q: You get excellent validation but poor production performance. Why?

A: Possible distribution drift, leakage, train/production mismatch, data-quality issues, or overly optimistic validation.

Q: A future feature is available only after the event. Can you use it?

A: No, not if the prediction is supposed to be made before that event. It creates leakage.

Q: Which model is best: Random Forest or XGBoost?

A: There is no universal winner. Compare them using appropriate validation, metrics, data size, latency, interpretability, and business constraints.

Q: Why start with a simple model?

A: It gives an interpretable baseline and tells you whether added complexity actually improves the problem.

Q: What if your model predicts probabilities poorly but ranks cases well?

A: Ranking metrics such as AUC may look good while calibration is poor. If probabilities drive decisions, evaluate calibration too.

Q: How do you explain an ML scenario when you don't know the exact algorithm?

A: Define the target, data, split, baseline, model family, metric, leakage risks, and validation first; reason from fundamentals rather than inventing details.

13. Interview Answer Pattern — Memorize the Structure, Not the Script

For almost any case study, use this order: **Problem** → **Target/X,y** → **Data quality** → **Split/leakage** → **Baseline** → **Features** → **Model candidates** → **Metric/business cost** → **Validation** → **Explainability/monitoring**.

The interviewer is often testing whether you can make sensible decisions and justify them, not whether you can name the fanciest algorithm.

For forecasting cases, explicitly mention time-based splitting and leakage. For imbalanced classification, explicitly discuss the confusion matrix and precision/recall trade-off. For high-impact decisions, mention explainability, fairness, calibration, and monitoring.

14. NLP / Deep Learning Checklist

- Tokenization converts raw text into tokens/IDs; embeddings provide learned vector representations.
- POS tagging gives grammatical categories; parsing captures syntactic relationships; coreference resolves mentions such as 'he' or 'it'.
- $\text{ReLU} = \max(0, z)$, $\tanh$ ranges roughly -1 to 1 , sigmoid is common for binary/independent multilabel outputs, and softmax is common for mutually exclusive multiclass outputs.
- ANNs are general neural networks; CNNs exploit local connectivity and weight sharing for spatial data.
- CNN pipeline: image → convolution → feature map → ReLU → pooling → more convolution → higher-level features → flatten/global pooling → dense/output.
- BERT is encoder-style and bidirectional for understanding; GPT is decoder-style causal/autoregressive generation.

15. HyperVerge Deepfake Project — Interview Mental Model

ROIs → RGB signals → SNR-weighted fusion → CHROM → rPPG → PSD maps → CNN → Real/Fake.

Core idea: real faces show periodic blood-flow-related color variation (rPPG); the project uses multi-ROI signal extraction and time-frequency representations to distinguish real/fake video signals.

Exact paper result: 96.13% accuracy and $F1 = 0.96$. The resume may state ~97%. The contribution was primarily signal processing/model development/training/evaluation; do not claim production deployment unless actually owned.

16. RAG / Vector Search Checklist

- RAG retrieves relevant evidence and provides it to the LLM so the answer is grounded in retrieved information.

- Vector databases specialize in storing embeddings and similarity search; FAISS is a vector-search library.
- HNSW is an ANN indexing algorithm using a multi-layer graph; efSearch controls search breadth.
- IVF partitions vectors into clusters; nprobe controls how many clusters are searched.
- RAG failure modes: not retrieved → improve chunking/embeddings/hybrid/query rewrite; retrieved but ignored → reduce noise/rerank/lost-in-middle handling; hallucination → stronger grounding/verification; conflict → freshness/deduplication; slow → reduce context/cache/batch/parallelize.
- HyDE generates a hypothetical answer/document for retrieval; the hypothetical text itself is not treated as truth.
- Faithfulness can be illustrated as supported claims divided by total claims; this is distinct from retrieval Recall@K.

17. Final Interview Reminders

- Start with problem formulation: target, input features, and learning type.
- State assumptions instead of presenting intuition as fact.
- For forecasting/time-dependent problems, explicitly think about time-based splitting and leakage.
- Choose metrics based on the business cost of errors.
- Separate predictive feature importance from causality.
- If you do not know a technique, reason from first principles instead of inventing an explanation.
- Scenario questions are often testing structured thinking: define → data → split → baseline → features → model → metrics → validation → monitoring.